

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №5
по дисциплине
«Функциональное программирование»
Вариант 11

Студент,
группы 5130201/30102

_____ Санько В. В.

Преподаватель,
к. т. н., доц.

_____ Моторин Д. Е.

« _____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
Постановка задачи	4
1 Реализация	5
1.1 Подключение расширений и импорт библиотек	5
1.2 Определение типов данных и стека монад	5
1.3 Логика проверки доступа	6
1.4 Загрузка конфигурации и вспомогательные функции	6
1.5 Главный цикл и обработка результатов	6
2 Тестирование	8
Заключение	10
Список литературы	11

Введение

Для решения задач, требующих совмещения доступа к конфигурации и обработки ошибок, в языке Haskell используются трансформеры монад. Трансформеры позволяют «наслаивать» функциональность различных монад друг на друга.

В данной работе реализована система авторизации, использующая стек трансформеров: **ReaderT** для передачи неизменяемой конфигурации системы и **ExceptT** для обработки исключительных ситуаций, таких как неверный токен или отсутствие прав доступа. Программа также интегрирована с библиотекой **aeson** для загрузки настроек из внешнего JSON-файла.

Постановка задачи

Цель работы: Разработать на языке Haskell программу аутентификации пользователя с использованием трансформеров монад и внешнего файла конфигурации.

Задачи:

- Описать структуру конфигурации системы и реализовать её чтение из формата JSON.
- Определить стек трансформеров `AuthStack` на базе `ExceptT`, `ReaderT` и `IO`.
- Реализовать логику проверки токена доступа и прав администратора.
- Обеспечить корректный вывод сообщений пользователю, учитывая буферизацию потока вывода.
- Обработать ошибки превышения количества попыток ввода и ограничение доступа к защищенным ресурсам.
- Реализовать проект с использованием сборщика `stack`.

1 Реализация

1.1 Подключение расширений и импорт библиотек

Для работы программы требуются расширения языка для обработки строковых литералов и автоматического вывода экземпляров классов типов.

```
1 {-# LANGUAGE OverloadedStrings #-}
2 {-# LANGUAGE DeriveGeneric #-}
3
4 module Main where
5
6 import Control.Monad.Reader (ReaderT, runReaderT, ask, lift)
7 import Control.Monad.Except (ExceptT, runExceptT, throwError)
8 import Control.Monad.IO.Class (MonadIO, liftIO)
9 import Data.Aeson (FromJSON, eitherDecodeFileStrict)
10 import Data.Text (Text)
11 import qualified Data.Text as T
12 import qualified Data.Text.IO as TIO
13 import GHC.Generics (Generic)
14 import System.Exit (exitSuccess)
15 import System.IO (hSetBuffering, BufferMode(NoBuffering), stdout)
```

Использование `OverloadedStrings` упрощает работу с типом `Text`, а `DeriveGeneric` позволяет библиотеке `aeson` автоматически сопоставлять поля JSON с полями структуры данных Haskell. Импорты охватывают все необходимые инструменты для работы со стеком трансформеров.

1.2 Определение типов данных и стека монад

Основой системы является конфигурация и типы возможных ошибок.

```
1 data Config = Config
2 { adminToken :: Text
3   , userTokens :: [Text]
4   , maxAttempts :: Int
5 } deriving (Show, Generic)
6
7 instance FromJSON Config
8
9 data AuthError
10 = InvalidToken
11 | NoAttemptsLeft
12 | ForbiddenResource
13 deriving (Show, Eq)
14
15 type AuthStack = ExceptT AuthError (ReaderT Config IO)
```

Тип `AuthStack` представляет собой стек, где `ExceptT` находится на верхнем уровне для возможности прерывания вычислений при ошибке, а `ReaderT` обеспечивает доступ к конфигурации на протяжении всей работы.

1.3 Логика проверки доступа

Основная функция проверки `checkAccess` работает внутри созданного стека монад.

```
1  checkAccess :: Text -> Bool -> AuthStack ()
2  checkAccess token isAdminPanel = do
3  config <- lift ask
4
5  if maxAttempts config <= 0
6  then throwError NoAttemptsLeft
7  else do
8  let isValid = token `elem` userTokens config || token == adminToken
9      config
10 if not isValid
11 then throwError InvalidToken
12 else if isAdminPanel && token /= adminToken config
13 then throwError ForbiddenResource
14 else return ()
```

Здесь используется функция `lift ask` для того, чтобы «поднять» действие чтения из внутреннего слоя `ReaderT` во внешний `ExceptT`. В случае обнаружения невалидных данных используется `throwError` для немедленного выхода из логической цепочки.

1.4 Загрузка конфигурации и вспомогательные функции

Программа поддерживает внешнее управление через `config.json`.

```
1  loadConfig :: FilePath -> IO Config
2  loadConfig path = do
3  result <- eitherDecodeFileStrict path
4  case result of
5  Left err -> do
6  putStrLn $ "config.json:␣" ++ err
7  return Config { adminToken = "secret123", userTokens = ["abc"],
8                  maxAttempts = 3 }
9  Right cfg -> return cfg
10
11 askUser :: Text -> IO Text
12 askUser prompt = T.putStr prompt >> T.getLine
13
14 isYes :: Text -> Bool
15 isYes input = T.toLower input `elem` ["", "yes", "y"]
```

Функция `loadConfig` обеспечивает отказоустойчивость: если файл отсутствует или содержит ошибки, система переходит на значения по умолчанию. Это гарантирует работоспособность программы в любых условиях.

1.5 Главный цикл и обработка результатов

Управление потоком выполнения реализовано через рекурсивный `authLoop`.

```

1  main :: IO ()
2  main = do
3    hSetBuffering stdout NoBuffering
4    config <- loadConfig "config.json"
5    authLoop config 0
6
7  authLoop :: Config -> Int -> IO ()
8  authLoop config failedAttempts = do
9    putStrLn "\n---"
10   token <- askUser ":"
11   adminInput <- askUser ":"
12
13   result <- runReaderT (runExceptT (checkAccess token (isYes adminInput)))
14     config
15
16   case result of
17     Right () -> putStrLn "\n"
18     Left err -> handleError err config failedAttempts
19
20  handleError :: AuthError -> Config -> Int -> IO ()
21  handleError err config failedAttempts = case err of
22    InvalidToken -> do
23      let newAttempts = failedAttempts + 1
24      if newAttempts >= maxAttempts config
25      then putStrLn "" >> exitSuccess
26      else authLoop config newAttempts
27    _ -> authLoop config failedAttempts

```

В функции `main` отключается буферизация (`NoBuffering`), что критично для корректного отображения запросов ввода. Запуск стека производится вложенными вызовами `runReaderT` и `runExceptT`.

2 Тестирование

Для проверки программы использовался файл `config.json` следующего содержания:

```
[language=json, numbers=none]
{
  "adminToken": "secret123",
  "userTokens": ["abc", "def"],
  "maxAttempts": 3
}
```

Пример работы программы при попытке входа в роли админа:

```
--- Новая попытка ---
Введите токен: adminload: 1.71  cmd:
oke
Нужна админ-панель? (да/нет): s
Неверный токен! Осталось попыток: 2

ОШИБКА: InvalidToken
Попробуйте еще раз (попытка 2 из 3)

--- Новая попытка ---
Введите токен: secret123
Нужна админ-панель? (да/нет): да
Проверка пройдена!

УСПЕХ: Доступ разрешен!
Вы вошли в админ-панель
```

Рис. 1: Вход с правами администратора

Пример работы программы при входе с неверными правами:

```
Введите токен: def
Нужна админ-панель? (да/нет): да
Токен не подходит для админ-панели

ОШИБКА: ForbiddenResource
У вас нет прав для админ-панели
Можете попробовать войти как обычный пользователь
```

Рис. 2: Попытка входа с пользователя в администратора

Пример работы программы при вводе неверного токена:

```
Введите токен: ппап
Нужна админ-панель? (да/нет): нет
Неверный токен! Осталось попыток: 2
```

```
ОШИБКА: InvalidToken
Попробуйте еще раз (попытка 2 из 3)
```

Рис. 3: Неверный токен

Пример успешного входа пользователя:

```
--- Новая попытка ---
Введите токен: abc
Нужна админ-панель? (да/нет): нет
Проверка пройдена!
```

```
УСПЕХ: Доступ разрешен!
Вы вошли как пользователь
```

Рис. 4: Успешный вход пользователя

Заключение

В ходе работы была разработана программа на языке Haskell, демонстрирующая применение трансформеров монад (`ReaderT` и `ExceptT`). Это позволило элегантно совместить чистую логику обработки ошибок с доступом к глобальной конфигурации, сохраняя при этом возможность выполнения побочных эффектов ввода-вывода. Также были освоены расширения языка (`DeriveGeneric`) и работа с библиотекой `aeson`.

Список литературы

- [1] Haskell Official Documentation – официальная документация по языку Haskell [Электронный ресурс]. <https://www.haskell.org/documentation/>
- [2] Миран Липовача. Изучай Haskell во имя добра! / Пер. с англ. Леушина Д., Синицына А., Арсанукаева Я. — М.: ДМК Пресс, 2012. — 490 с.
- [3] Официальная документация Haskell: <https://www.haskell.org> [Дата обращения: 17.11.2025]